Weather Trend Forecasting — Tech Assessment

Objective: Analyze the *Global Weather Repository* to forecast weather trends and document data-science workflow (cleaning, EDA, modeling, advanced topics).

Dataset: World Weather Repository on Kaggle — daily snapshot with 40+ features; the assessment asks to use `last_updated` as the time axis for time-series work.

PM Accelerator mission (submission)

Our Mission

By making industry-leading tools and education available to individuals from all backgrounds, we **level the playing field** for future PM leaders. This is the PM Accelerator motto: we grant aspiring and experienced PMs what they need most — **Access**. We introduce you to industry leaders, surround you with the right PM ecosystem, and discover the **new world of AI product management** skills.

- PM Accelerator

How to run this notebook

1. `pip install -r requirements.txt`
2. Download data (cell below uses **KaggleHub**), or place `GlobalWeatherRepository.csv` in a `data/` folder next to this notebook, or set `WEATHER_CSV_PATH` to the file.
3. **Run all cells** from top to bottom.

0. Imports and configuration

In [1]:
```
import os
import warnings
from pathlib import Path
from typing import Any

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
warnings.filterwarnings("ignore", category=UserWarning)
warnings.filterwarnings("ignore", category=FutureWarning)
sns.set_theme(style="whitegrid", context="notebook")
plt.rcParams["figure.figsize"] = (10, 4)

NUMERIC_FEATURES = [
    "temperature_celsius", "precip_mm", "humidity", "wind_kph", "pressure_mb", "clo
    "uv_index", "visibility_km", "air_quality_PM2.5", "air_quality_PM10",
    "air_quality_Carbon_Monoxide", "air_quality_Ozone", "latitude", "longitude",
]
```

Download data (KaggleHub — assessment snippet)

If the CSV is not already local, the next cell downloads the dataset. Requires network access on first run.

In [2]:
```
def resolve_csv_path() -> Path:
    """Local env → ./data/ → parent ./data/ → KaggleHub."""
    override = os.environ.get("WEATHER_CSV_PATH")
    if override and Path(override).is_file():
        return Path(override)
    for base in (Path.cwd(), Path.cwd().parent):
        p = base / "data" / "GlobalWeatherRepository.csv"
        if p.is_file():
            return p
    import kagglehub

    root = Path(kagglehub.dataset_download("nelgiriyewithana/global-weather-reposit
    csv_path = root / "GlobalWeatherRepository.csv"
    if not csv_path.is_file():
        raise FileNotFoundError(f"No CSV under {root}")
    return csv_path


CSV_PATH = resolve_csv_path()
print("Using CSV:", CSV_PATH)
```

```
Using CSV: C:\Users\eydos\.cache\kagglehub\datasets\nelgiriyewithana\global-weather-
repository\versions\913\GlobalWeatherRepository.csv
```

Helper functions (cleaning, forecasting, advanced analysis)

Everything below is self-contained in this notebook — no separate Python modules.

In [3]:
```
def load_raw(csv_path: Path | None = None) -> pd.DataFrame:
    path = csv_path or CSV_PATH
    df = pd.read_csv(path, low_memory=False)
    df["last_updated"] = pd.to_datetime(df["last_updated"], errors="coerce")
    return df


def clean_frame(df: pd.DataFrame) -> pd.DataFrame:
    out = df.dropna(subset=["last_updated"]).copy()
```

```
    for c in NUMERIC_FEATURES:
        if c in out.columns:
            out[c] = pd.to_numeric(out[c], errors="coerce")
    for c in NUMERIC_FEATURES:
        if c in out.columns:
            out[c] = out[c].fillna(out[c].median())
    for col in ["temperature_celsius", "precip_mm"]:
        if col not in out.columns:
            continue
        q1, q3 = out[col].quantile([0.25, 0.75])
        iqr = q3 - q1
        lo, hi = q1 - 3 * iqr, q3 + 3 * iqr
        out[col] = out[col].clip(lo, hi)
    return out


def add_climate_zone(lat: float) -> str:
    a = abs(float(lat))
    if a < 23.5:
        return "Tropical"
    if a < 35:
        return "Subtropical"
    if a < 55:
        return "Temperate"
    return "Cold/Boreal"


def enrich_for_analysis(df: pd.DataFrame) -> pd.DataFrame:
    out = df.copy()
    out["climate_zone"] = out["latitude"].apply(add_climate_zone)
    return out


def city_series(df: pd.DataFrame, city: str) -> pd.DataFrame:
    g = df[df["location_name"] == city].sort_values("last_updated").copy()
    return g.set_index("last_updated")


def train_test_split_ts(y: pd.Series, test_ratio: float = 0.2):
    cut = int(len(y) * (1 - test_ratio))
    return y.iloc[:cut], y.iloc[cut:]


def build_supervised_matrix(series_df: pd.DataFrame, target: str = "temperature_cel
    lags = lags or [1, 2, 3, 7]
    s = series_df[[target, "precip_mm", "humidity"]].copy()
    for L in lags:
        s[f"lag_{L}"] = s[target].shift(L)
    s["roll3"] = s[target].rolling(3).mean()
    s = s.dropna()
    feat_cols = [c for c in s.columns if c != target]
    return s[feat_cols], s[target]


def evaluate_forecast(y_true, y_pred):
    y_true = np.asarray(y_true, dtype=float)
```

```
    y_pred = np.asarray(y_pred, dtype=float)
    mae = float(np.mean(np.abs(y_true - y_pred)))
    rmse = float(np.sqrt(np.mean((y_true - y_pred) ** 2)))
    mask = y_true != 0
    mape = float(np.mean(np.abs((y_true[mask] - y_pred[mask]) / y_true[mask])) * 10
    return {"MAE": mae, "RMSE": rmse, "MAPE": mape}


def forecast_naive_seasonal(y_train: pd.Series, h: int, season: int = 1):
    if len(y_train) <= season:
        return np.full(h, y_train.iloc[-1])
    return np.full(h, y_train.iloc[-season])


def forecast_arima(y_train: pd.Series, h: int):
    try:
        from statsmodels.tsa.arima.model import ARIMA

        fit = ARIMA(y_train, order=(2, 1, 2), trend="n").fit()
        return np.asarray(fit.forecast(steps=h), dtype=float)
    except Exception:
        return None


def run_city_forecast_experiment(df: pd.DataFrame, city: str) -> dict[str, Any]:
    g = city_series(df, city)
    if len(g) < 80:
        raise ValueError(f"Not enough history for {city}")
    y = g["temperature_celsius"].resample("1D").mean().interpolate("time").dropna()
    y_train, y_test = train_test_split_ts(y, 0.2)
    h = len(y_test)
    preds = {"naive_lag1": forecast_naive_seasonal(y_train, h, season=1)}
    ar = forecast_arima(y_train, h)
    if ar is not None:
        preds["ARIMA(2,1,2)"] = ar
    num_cols = [c for c in ("temperature_celsius", "precip_mm", "humidity") if c in
    g_aligned = g[num_cols].resample("1D").mean().reindex(y.index).interpolate("tim
    X, y_all = build_supervised_matrix(g_aligned)
    cut_idx = y_train.index[-1]
    train_mask = y_all.index <= cut_idx
    test_mask = y_all.index.isin(y_test.index)
    X_tr, y_tr = X.loc[train_mask], y_all.loc[train_mask]
    X_te = X.loc[test_mask]
    fi = None
    if len(X_te) == h and len(X_tr) > 30:
        from sklearn.ensemble import RandomForestRegressor

        rf = RandomForestRegressor(n_estimators=200, max_depth=12, random_state=42,
        rf.fit(X_tr, y_tr)
        preds["RandomForest_lags"] = rf.predict(X_te)
        fi = pd.Series(rf.feature_importances_, index=X_tr.columns).sort_values(asc
    y_true = y_test.values
    rows = []
    ensemble_comps = []
    for name, p in preds.items():
        if p is None or len(p) != len(y_true):
```

```
            continue
        m = evaluate_forecast(y_true, p)
        m["model"] = name
        rows.append(m)
        if name != "naive_lag1":
            ensemble_comps.append(p)
    if ensemble_comps:
        ens = np.mean(np.vstack(ensemble_comps), axis=0)
        m = evaluate_forecast(y_true, ens)
        m["model"] = "Ensemble (mean of ARIMA+RF)"
        rows.append(m)
    return {
        "city": city,
        "y_train": y_train,
        "y_test": y_test,
        "preds": preds,
        "metrics": pd.DataFrame(rows),
        "rf_feature_importance": fi,
        "y_true": y_true,
    }


def anomaly_scores(df: pd.DataFrame, sample: int | None = 8000):
    from sklearn.ensemble import IsolationForest
    from sklearn.preprocessing import StandardScaler

    cols = [c for c in NUMERIC_FEATURES if c in df.columns]
    sub = df[cols].dropna()
    if sample and len(sub) > sample:
        sub = sub.sample(sample, random_state=42)
    Xs = StandardScaler().fit_transform(sub)
    iso = IsolationForest(contamination=0.02, random_state=42, n_jobs=-1)
    iso.fit(Xs)
    sub = sub.copy()
    sub["anomaly_score"] = iso.decision_function(Xs)
    sub["is_anomaly"] = iso.predict(Xs) == -1
    return sub, iso


def air_quality_correlation(df: pd.DataFrame) -> pd.DataFrame:
    aq = [c for c in df.columns if c.startswith("air_quality_") and df[c].dtype !=
    weather = ["temperature_celsius", "humidity", "wind_kph", "precip_mm", "pressur
    cols = [c for c in aq + weather if c in df.columns]
    return df[cols].corr(method="spearman")


def monthly_climate_trends(df: pd.DataFrame) -> pd.DataFrame:
    x = df.copy()
    x["month"] = x["last_updated"].dt.to_period("M").dt.to_timestamp()
    return x.groupby("month").agg(
        mean_temp=("temperature_celsius", "mean"),
        mean_precip=("precip_mm", "mean"),
        n=("location_name", "nunique"),
    )
```

```
def country_summary(df: pd.DataFrame) -> pd.DataFrame:
    return (
        df.groupby("country")
        .agg(
            mean_temp=("temperature_celsius", "mean"),
            std_temp=("temperature_celsius", "std"),
            mean_pm25=("air_quality_PM2.5", "mean"),
            cities=("location_name", "nunique"),
        )
        .sort_values("mean_temp", ascending=False)
    )
```

---

1. Data cleaning & preprocessing

Step	Rationale
Drop rows with missing `last_updated`	Time series requires a valid timestamp
Coerce numerics; fill NaNs with **column medians**	Simple baseline imputation
IQR soft caps (3×IQR) on temperature & precipitation	Reduce impact of extreme outliers / bad readings
Add `climate_zone` from latitude	Supports regional / climate analysis

Normalization note: `StandardScaler` is used later only for **IsolationForest**. Tree models use raw lag features; that is normal practice.

In [4]:
```
df_raw = load_raw()
df = enrich_for_analysis(clean_frame(df_raw))
print("Shape:", df.shape)
df.head(3)
```

```
Shape: (133318, 42)
```

Out[4]:

	country	location_name	latitude	longitude	timezone	last_updated_epoch	last_
0	Afghanistan	Kabul	34.52	69.18	Asia/Kabul	1715849100	202
1	Albania	Tirana	41.33	19.82	Europe/Tirane	1715849100	202
2	Algeria	Algiers	36.76	3.05	Africa/Algiers	1715849100	202

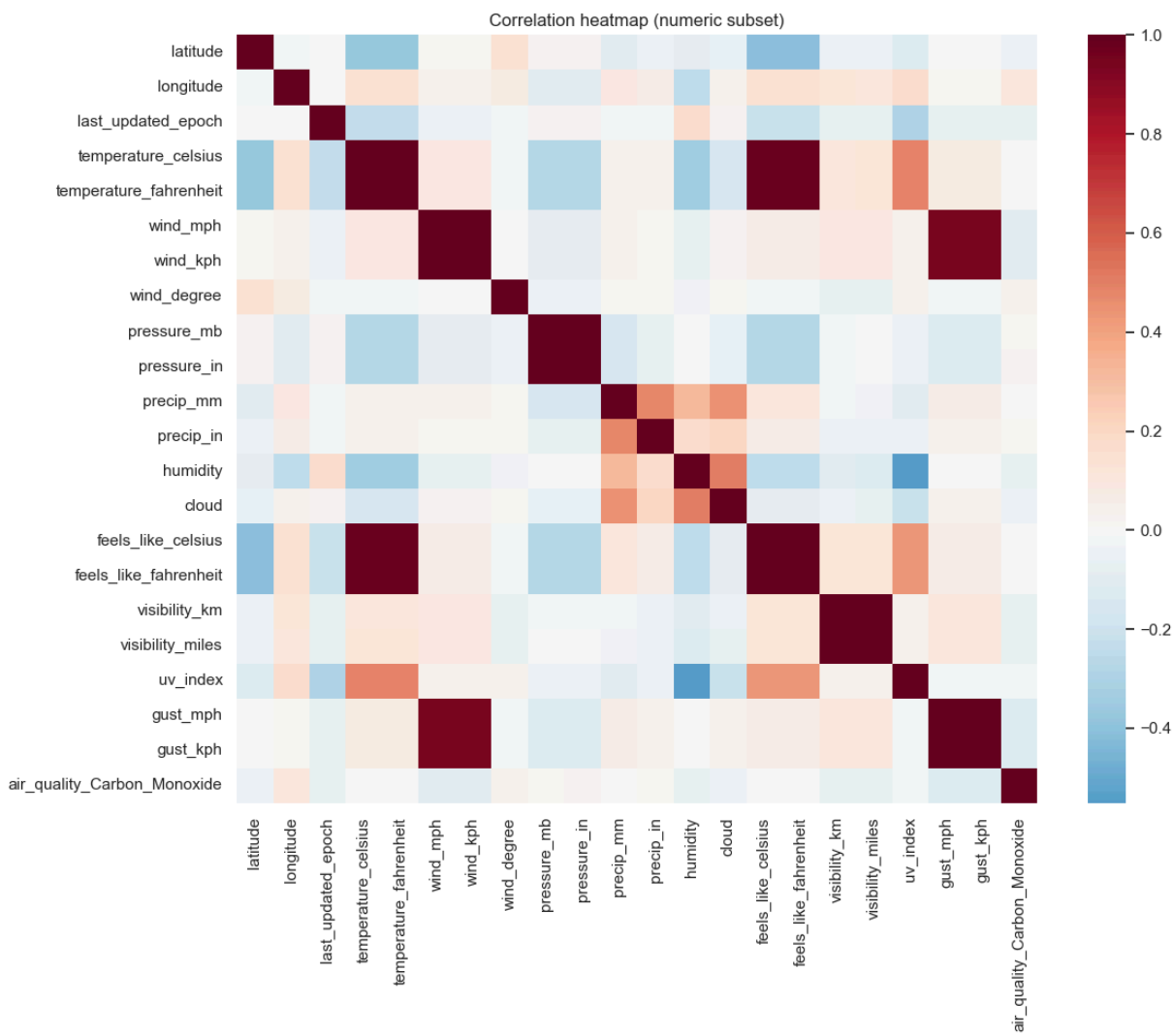
3 rows × 42 columns

---

2. Exploratory Data Analysis (EDA)

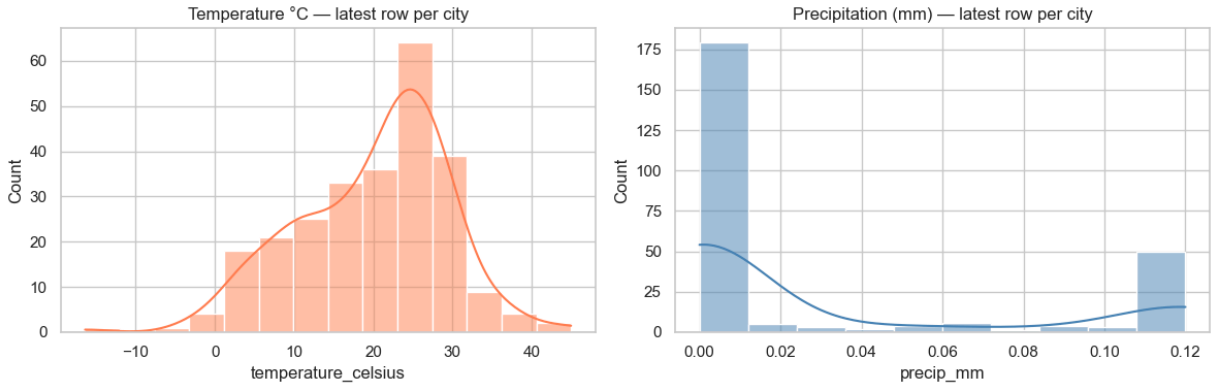
- **Correlations** among numeric weather and air-quality fields
- **Distributions** of temperature and precipitation (latest observation per city = cross-sectional snapshot)
- **Global mean temperature** over calendar time (all cities pooled by day)

In [5]:
```
num = [c for c in df.select_dtypes(include=np.number).columns if df[c].notna().sum(
plt.figure(figsize=(12, 10))
sns.heatmap(df[num].corr(), center=0, cmap="RdBu_r")
plt.title("Correlation heatmap (numeric subset)")
plt.tight_layout()
plt.show()
```

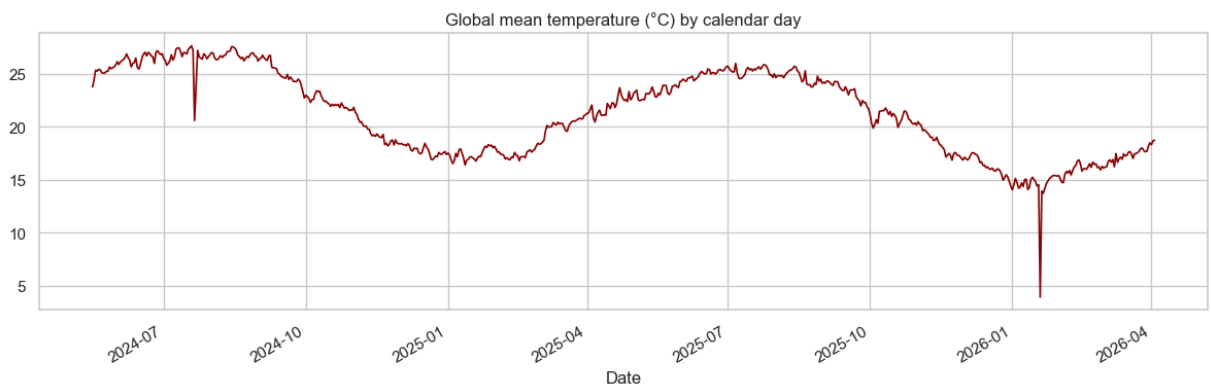


In [6]:
```
latest = df.sort_values("last_updated").groupby("location_name", as_index=False).ta
fig, ax = plt.subplots(1, 2, figsize=(12, 4))
sns.histplot(latest["temperature_celsius"], kde=True, ax=ax[0], color="coral")
ax[0].set_title("Temperature °C — latest row per city")
sns.histplot(latest["precip_mm"], kde=True, ax=ax[1], color="steelblue")
ax[1].set_title("Precipitation (mm) — latest row per city")
```

```
plt.tight_layout()
plt.show()
```



In [7]:
```
daily_global = df.groupby(df["last_updated"].dt.normalize())["temperature_celsius"]
fig, ax = plt.subplots(figsize=(12, 4))
daily_global.plot(ax=ax, color="darkred", lw=1.2)
ax.set_title("Global mean temperature (°C) by calendar day")
ax.set_xlabel("Date")
plt.tight_layout()
plt.show()
```



---

3. Forecasting (time series on `last_updated`)

Setup: Pick a city with long history. Resample to **one value per calendar day** (daily mean) so timestamps align cleanly. **Chronological split:** 80% train / 20% test (no shuffling).

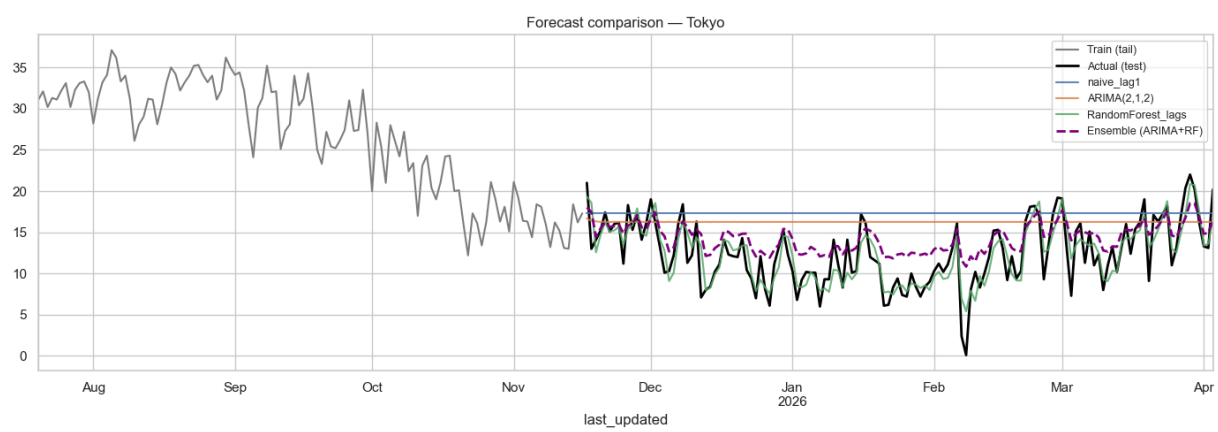
Model	Description
Naive (lag-1)	Repeat last training value — baseline
ARIMA(2,1,2)	Classical univariate model (`statsmodels`)
RandomForest	Lags of temperature + precip + humidity + 3-day rolling mean
Ensemble	Mean of ARIMA and RF (naive excluded so it does not dilute the blend)

Metrics: MAE, RMSE, MAPE. *MAPE can behave poorly when temperatures are near 0°C.*

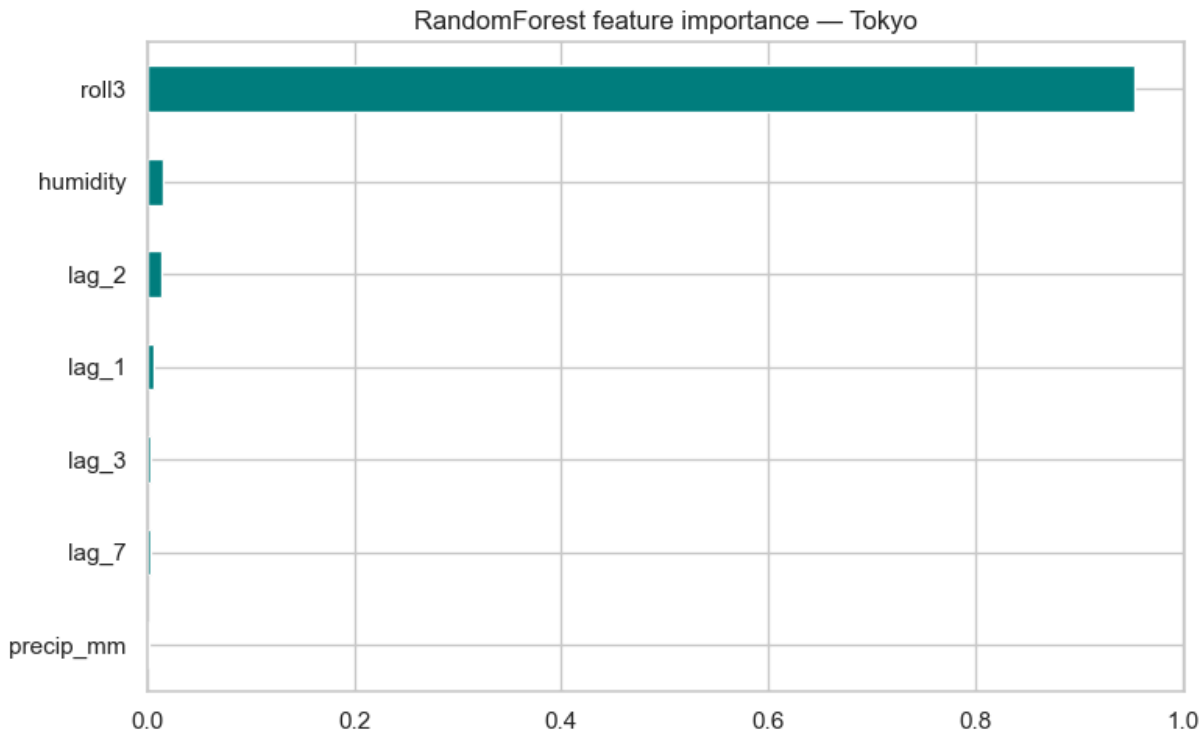
In [8]:
```
FORECAST_CITY = "Tokyo"
exp = run_city_forecast_experiment(df, FORECAST_CITY)
display(exp["metrics"])
```

	MAE	RMSE	MAPE	model
0	5.183333	6.186187	179.861997	naive_lag1
1	4.456035	5.421762	164.807027	ARIMA(2,1,2)
2	1.561808	1.969534	52.649364	RandomForest_lags
3	2.556949	3.170769	104.907223	Ensemble (mean of ARIMA+RF)

In [9]:
```
fig, ax = plt.subplots(figsize=(14, 5))
exp["y_train"].iloc[-120:].plot(ax=ax, label="Train (tail)", color="gray")
exp["y_test"].plot(ax=ax, label="Actual (test)", color="black", lw=2)
h, idx = len(exp["y_test"]), exp["y_test"].index
for name, p in exp["preds"].items():
    if p is not None and len(p) == h:
        ax.plot(idx, p, label=name, alpha=0.85)
if "ARIMA(2,1,2)" in exp["preds"] and "RandomForest_lags" in exp["preds"]:
    ax.plot(idx, (exp["preds"]["ARIMA(2,1,2)"] + exp["preds"]["RandomForest_lags"])
ax.set_title(f"Forecast comparison — {FORECAST_CITY}")
ax.legend(loc="best", fontsize=9)
plt.tight_layout()
plt.show()
```



In [10]:
```
fi = exp.get("rf_feature_importance")
if fi is not None:
    plt.figure(figsize=(8, 5))
    fi.head(12).iloc[::-1].plot(kind="barh", color="teal")
    plt.title(f"RandomForest feature importance — {FORECAST_CITY}")
    plt.tight_layout()
    plt.show()
else:
    print("No RF feature importance (insufficient training rows).")
```



---

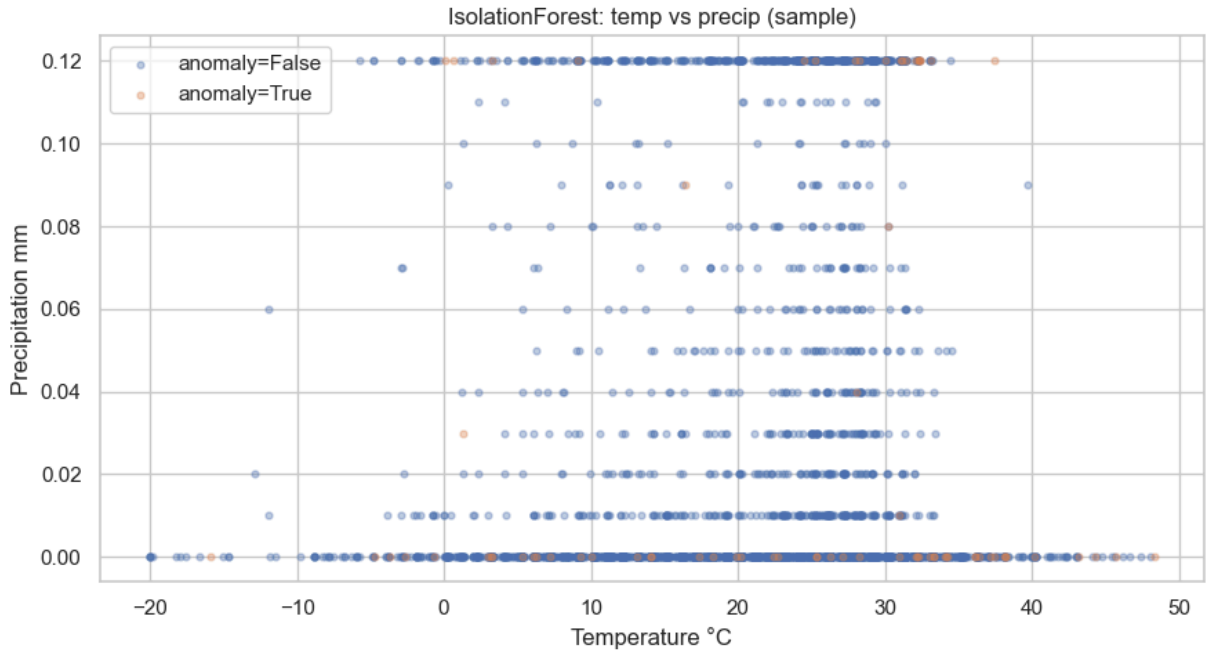
4. Advanced analyses

4.1 Anomaly detection

IsolationForest on scaled numeric weather/air-quality features. Points flagged as anomalies may reflect rare weather combinations or data quirks — interpret with domain context.

In [11]:
```
sub, _ = anomaly_scores(df, sample=6000)
print("Flagged anomaly rate:", round(sub["is_anomaly"].mean() * 100, 2), "%")
sample_plot = sub.reset_index(drop=True).sample(min(4000, len(sub)), random_state=0
plt.figure(figsize=(9, 5))
for flag, g in sample_plot.groupby("is_anomaly"):
    plt.scatter(g["temperature_celsius"], g["precip_mm"], alpha=0.35, s=12, label=f
plt.xlabel("Temperature °C")
plt.ylabel("Precipitation mm")
plt.title("IsolationForest: temp vs precip (sample)")
plt.legend()
plt.tight_layout()
plt.show()
```

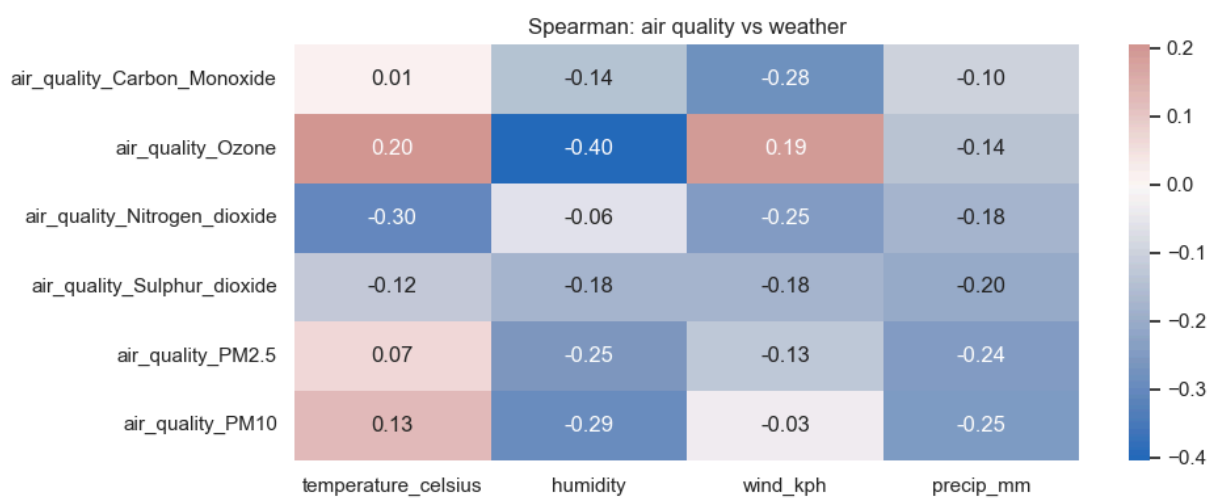
```
Flagged anomaly rate: 2.0 %
```



4.2 Environmental impact — air quality vs weather

Spearman correlation (robust to monotone non-linear relationships) between air-quality columns and selected weather variables.

In [12]:
```
aq_full = air_quality_correlation(df)
aq_cols = [c for c in aq_full.columns if c.startswith("air_quality_")][:6]
wx_cols = [c for c in ["temperature_celsius", "humidity", "wind_kph", "precip_mm"]
if aq_cols and wx_cols:
    plt.figure(figsize=(10, 4))
    sns.heatmap(aq_full.loc[aq_cols, wx_cols], annot=True, fmt=".2f", cmap="vlag",
    plt.title("Spearman: air quality vs weather")
    plt.tight_layout()
    plt.show()
display(aq_full.iloc[:8, :6])
```



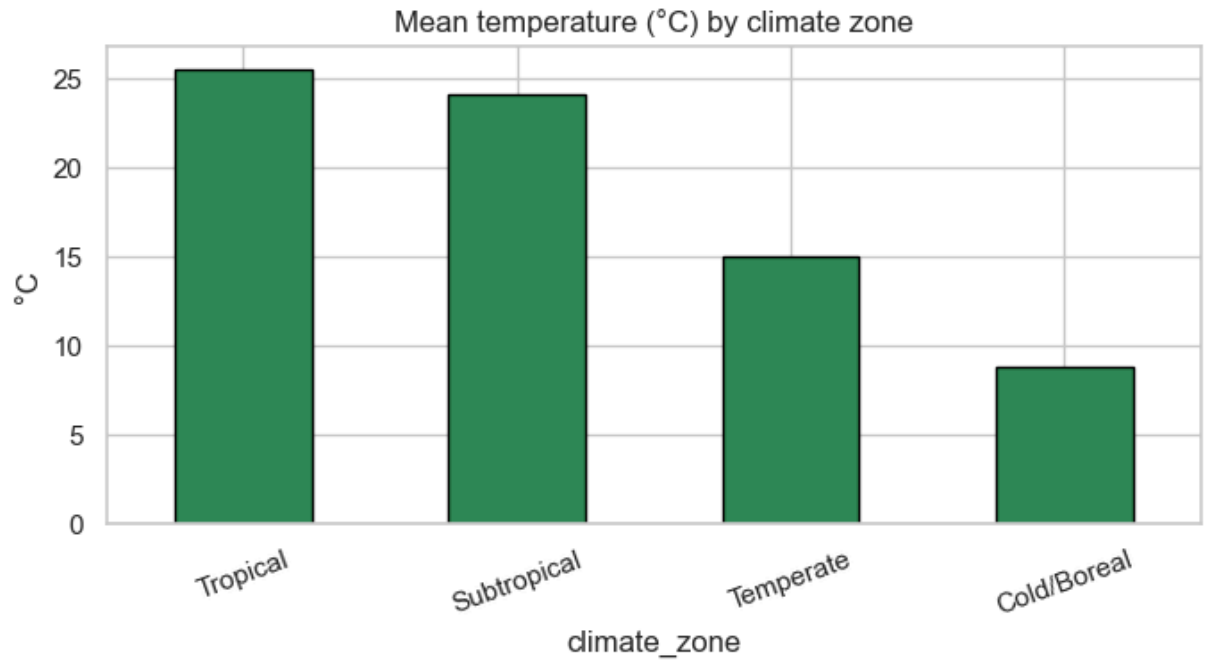
	air_quality_Carbon_Monoxide	air_quality_Ozone	air_quality_Nitr
air_quality_Carbon_Monoxide	1.000000	0.015590	
air_quality_Ozone	0.015590	1.000000	
air_quality_Nitrogen_dioxide	0.570861	-0.175327	
air_quality_Sulphur_dioxide	0.542654	0.023435	
air_quality_PM2.5	0.595657	0.098885	
air_quality_PM10	0.505912	0.146983	
air_quality_us-epa-index	0.560199	0.069614	
air_quality_gb-defra-index	0.579968	0.079813	

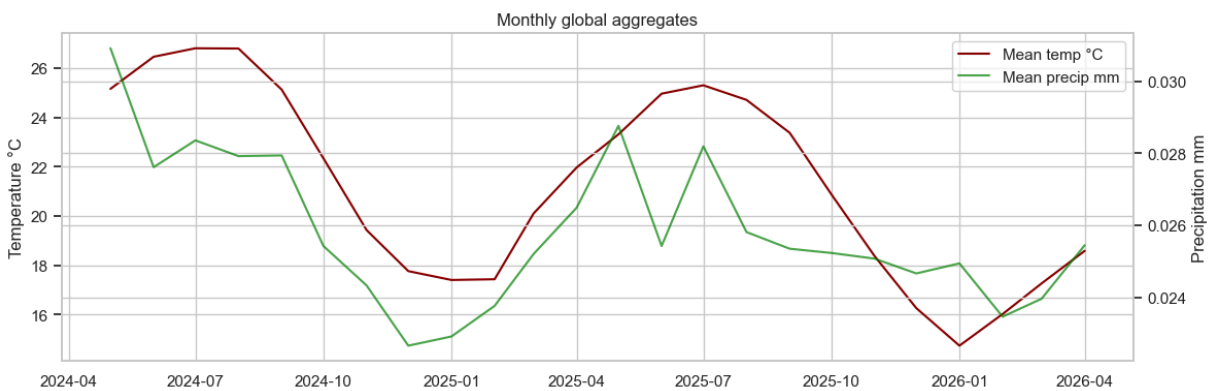
4.3 Climate patterns

Mean temperature by **latitude-based climate zone**, and **monthly** global mean temperature / precipitation.

In [13]:
```
zone = df.groupby("climate_zone")["temperature_celsius"].mean()
zone = zone.reindex(["Tropical", "Subtropical", "Temperate", "Cold/Boreal"]).dropna
plt.figure(figsize=(7, 4))
zone.plot(kind="bar", color="seagreen", edgecolor="black")
plt.title("Mean temperature (°C) by climate zone")
plt.ylabel("°C")
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()

clim = monthly_climate_trends(df)
fig, ax = plt.subplots(figsize=(12, 4))
ax.plot(clim.index, clim["mean_temp"], color="darkred", label="Mean temp °C")
ax.set_ylabel("Temperature °C")
ax2 = ax.twinx()
ax2.plot(clim.index, clim["mean_precip"], color="green", alpha=0.7, label="Mean pre
ax2.set_ylabel("Precipitation mm")
ax.set_title("Monthly global aggregates")
fig.legend(loc="upper right", bbox_to_anchor=(1, 1), bbox_transform=ax.transAxes)
plt.tight_layout()
plt.show()
```



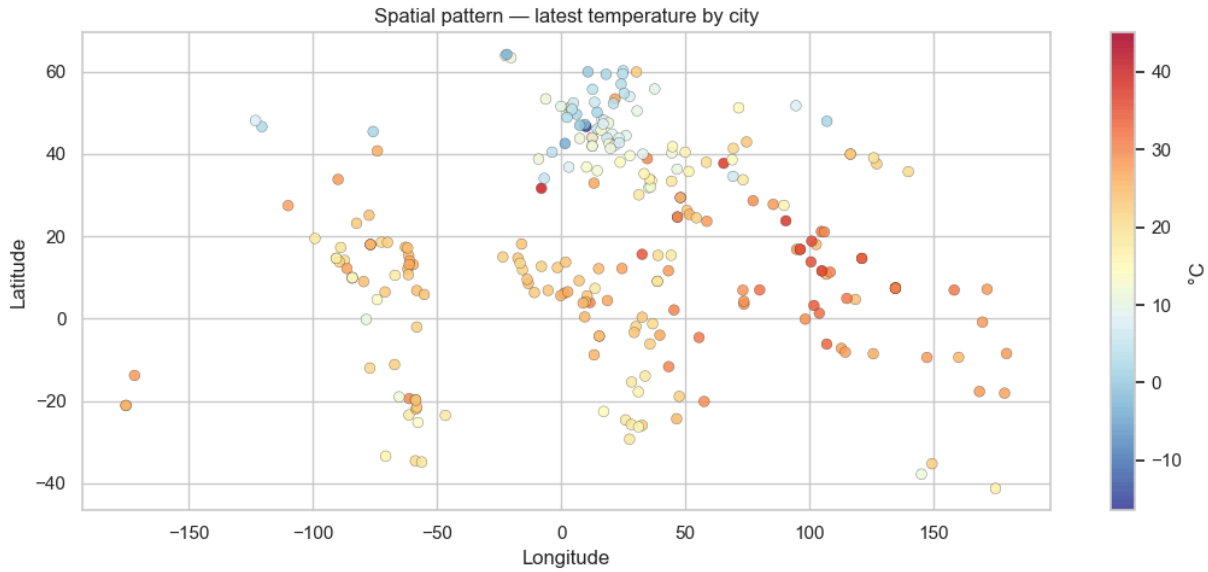


4.4 Spatial & geographic patterns

Scatter of **latitude / longitude** colored by temperature (latest snapshot per city). **Country-level** summary statistics.

In [14]:
```
plt.figure(figsize=(11, 5))
sc = plt.scatter(
    latest["longitude"],
    latest["latitude"],
    c=latest["temperature_celsius"],
    cmap="RdYlBu_r",
    s=40,
    alpha=0.85,
    edgecolor="k",
    linewidth=0.2,
)
plt.colorbar(sc, label="°C")
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.title("Spatial pattern — latest temperature by city")
```

```
plt.tight_layout()
plt.show()
```



In [15]:
```
cs = country_summary(df)
print("Top 12 countries by mean temperature:")
display(cs.head(12))
print("Bottom 8 countries by mean temperature:")
display(cs.tail(8))
```

```
Top 12 countries by mean temperature:
```

country	mean_temp	std_temp	mean_pm25	cities
Saudi Arabien	45.000000	NaN	22.300000	1
Marrocos	40.300000	NaN	8.200000	1
Turkménistan	37.800000	NaN	6.700000	1
Турция	34.000000	NaN	11.300000	1
United Arab Emirates	32.255620	6.985448	58.198182	1
Qatar	32.188287	7.826937	57.499473	1
Cambodia	31.491520	2.929746	26.281323	2
Djibouti	31.398829	4.959765	29.037606	1
Oman	31.362044	5.043598	36.082451	1
Inde	31.200000	NaN	34.600000	1
Kuwait	30.946053	10.897724	92.616637	2
Thailand	30.893212	3.118969	28.792481	2

```
Bottom 8 countries by mean temperature:
```

country	mean_temp	std_temp	mean_pm25	cities
Finland	8.643504	10.455749	7.505212	1
United States of America	8.559970	3.910630	6.137873	3
Russia	8.448704	12.903278	41.203070	4
Kazakhstan	7.996784	14.620416	12.101257	1
Norway	7.923572	8.056286	10.290798	1
Iceland	5.555783	4.642094	3.910116	3
Canada	4.836896	11.713551	8.914861	1
Mongolia	3.609942	15.522977	22.867463	1

---

5. Insights & limitations (summary)

- **Data:** Each row is a city snapshot; `last_updated` drives the time axis. Per-city series are resampled to **daily** frequency for modeling.
- **Cleaning:** Median imputation and IQR caps are transparent baselines; production pipelines might use per-city imputation or explicit missingness models.
- **Models:** RandomForest on lags often **outperforms** naive and sometimes ARIMA on short multivariate signals; the **ensemble** averages structured models (ARIMA smoothness + RF nonlinear patterns).
- **Air quality:** Correlations are **associative**, not causal — confounding by geography and season is likely.
- **Anomalies:** IsolationForest highlights **multivariate** outliers; validate flagged cases before treating them as errors.

Deliverable: Export this notebook to **HTML** or **PDF** (`File → Save and Export`) for a standalone report, or push the `.ipynb` to your public GitHub repo with this `README.md`.